

733. Flood Fill Easy

Acceptance: 67.7% | [LeetCode](#)

You are given an image represented by an $m \times n$ grid of integers `image`, where `image[i][j]` represents the pixel value of the image. You are also given three integers `sr`, `sc`, and `color`. Your task is to perform a **flood fill** on the image starting from the pixel `image[sr][sc]`.

To perform a **flood fill** :

1. Begin with the starting pixel and change its color to `color`.
2. Perform the same process for each pixel that is **directly adjacent** (pixels that share a side with the original pixel, either horizontally or vertically) and shares the **same color** as the starting pixel.
3. Keep **repeating** this process by checking neighboring pixels of the *updated* pixels and modifying their color if it matches the original color of the starting pixel.
4. The process **stops** when there are **no more** adjacent pixels of the original color to update.

Return the **modified** image after performing the flood fill.

Example 1:

Input: `image = [[1,1,1],[1,1,0],[1,0,1]]`, `sr = 1`, `sc = 1`, `color = 2`

Output: `[[2,2,2],[2,2,0],[2,0,1]]`

Explanation:

From the center of the image with position `(sr, sc) = (1, 1)` (i.e., the red pixel), all pixels connected by a path of the same color as the starting pixel (i.e., the blue pixels) are colored with the new color.

Note the bottom corner is **not** colored 2, because it is not horizontally or vertically connected to the starting pixel.

Example 2:

Input: `image = [[0,0,0],[0,0,0]]`, `sr = 0`, `sc = 0`, `color = 0`

Output: `[[0,0,0],[0,0,0]]`

Explanation:

The starting pixel is already colored with 0, which is the same as the target color. Therefore, no changes are made to the image.

Constraints:

- `m == image.length`
- `n == image[i].length`
- `1 <= m, n <= 50`
- `0 <= image[i][j], color < 216`
- `0 <= sr < m`
- `0 <= sc < n`

Claude Opus Solution

Flood Fill - Complete Solution

1. Intuition

This is a classic **graph traversal** problem disguised as an image problem. Think of the grid as a graph where: - Each pixel is a **node** - Adjacent pixels (up/down/left/right) sharing the **same color** are connected by **edges**

The flood fill operation is essentially finding all nodes in a **connected component** (pixels reachable from the start that share the original color) and recoloring them. This is exactly what **DFS or BFS** does!

Key Insight #1: We must remember the *original* color before we start changing anything, because we use it to decide which neighbors to fill.

Key Insight #2 (Critical Edge Case): If the new `color` equals the original color, we must return early. Otherwise, we'd enter an infinite loop—we keep "filling" pixels that are already the target color, never marking them as visited.

2. Approach (Pseudocode)

1. Record `original_color = image[sr][sc]`
2. If `original_color == color`:
 return `image` (no work needed, avoids infinite loop)
3. Run DFS from `(sr, sc)`:
 - a. If `current cell == original_color`:
 - Recolor it to `color` (this also marks it "visited")
 - Recurse into all 4 valid neighbors
4. Return `image`

The trick that avoids needing a separate `visited` set: **recoloring the cell itself marks it visited**, since it no longer matches `original_color`.

3. Python Solution

```
from typing import List

class Solution:
    def floodFill(self, image: List[List[int]], sr: int, sc: int, color: int) -> List[List[int]]:
        rows, cols = len(image), len(image[0])

        # The color we are "replacing". Captured BEFORE any modification.
        original_color = image[sr][sc]

        # CRUCIAL edge case: if the new color is the same as the original,
        # filling would never change cells, so DFS would recurse infinitely
        # (cells never get "marked" by changing). Just return as-is.
        if original_color == color:
            return image

        def dfs(r: int, c: int) -> None:
            # Bounds check: ignore cells outside the grid.
            if r < 0 or r >= rows or c < 0 or c >= cols:
                return
            # Only fill cells that match the ORIGINAL color.
            # Cells already changed to `color` won't match, so they act
            # as "visited" markers and stop the recursion naturally.
            if image[r][c] != original_color:
                return

            # Recolor the current cell (this also marks it as visited).
            image[r][c] = color

            # Explore the 4 directly-adjacent neighbors.
            dfs(r + 1, c) # down
            dfs(r - 1, c) # up
            dfs(r, c + 1) # right
            dfs(r, c - 1) # left

        # Start the flood fill from the given source pixel.
        dfs(sr, sc)
        return image
```

BFS Alternative (avoids recursion depth limits)

```
from collections import deque
from typing import List

class Solution:
    def floodFill(self, image: List[List[int]], sr: int, sc: int, color: int) -> List[List[int]]:
        rows, cols = len(image), len(image[0])
        original_color = image[sr][sc]

        # Same early-return guard against infinite loops.
        if original_color == color:
            return image

        # Use a queue for iterative breadth-first traversal.
        queue = deque([(sr, sc)])
        image[sr][sc] = color # recolor source immediately to mark visited

        while queue:
            r, c = queue.popleft()
            # Check all 4 directions.
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                nr, nc = r + dr, c + dc
                # Valid, in-bounds neighbor that still has the original color.
                if 0 <= nr < rows and 0 <= nc < cols and image[nr][nc] == original_color:
                    image[nr][nc] = color # recolor + mark visited
                    queue.append((nr, nc))

        return image
```

Note on the BFS version: It's important to recolor a cell **when you enqueue it**, not when you dequeue it. Otherwise, the same cell could be added to the queue multiple times by different neighbors before being processed (the reference `my_solution.py` recolors on dequeue, which can cause duplicate queue entries—still correct, but less efficient).

4. Complexity Analysis

Metric	Complexity	Explanation
Time	$O(m \times n)$	In the worst case (entire grid is one color), we visit every cell exactly once. Each cell does $O(1)$ work (4 neighbor checks).
Space (DFS)	$O(m \times n)$	Recursion call stack. Worst case is a "snake" path or full grid where the recursion depth reaches all cells.
Space (BFS)	$O(m \times n)$	The queue can hold up to all cells in the worst case.

Where `m` = number of rows, `n` = number of columns.

5. Alternative Approaches

- DFS (Recursive)** — *shown above*. Most concise. Risk: stack overflow on very large grids (not an issue here since `m, n ≤ 50`, max depth = 2500).
- BFS (Iterative with Queue)** — *shown above*. Safer for large inputs (no recursion limit), uses an explicit queue.
- DFS (Iterative with Stack)** — Same logic as recursive DFS but uses an explicit stack to avoid recursion: `python stack = [(sr, sc)] while stack: r, c = stack.pop() if 0 <= r < rows and 0 <= c < cols and image[r][c] == original_color: image[r][c] = color stack.extend([(r+1,c),(r-1,c),(r,c+1),(r,c-1)])`
- Union-Find** — Overkill here, but applicable to "number of islands"-style problems. Not efficient for a single fill operation.

Key Takeaways / Pattern Recognition

- Grid traversal = graph traversal.** Whenever you fill/count/explore connected regions in a 2D grid, reach for DFS or BFS.
- In-place recoloring = visited marker.** A common trick to save $O(m \times n)$ space on a `visited` array.
- Always check for the "no-op" edge case** (`original == target`) when the visited-mark and the fill use the same value—a frequent source of infinite loops.

Codex Solution

1. Intuition

This is a classic **graph traversal** problem.

Think of each pixel as a node in a graph. Each pixel is connected to its 4-directional neighbors:

- Up
- Down
- Left
- Right

Starting from `(sr, sc)`, we need to visit every pixel that:

1. Is connected to the starting pixel through 4-directional moves
2. Has the same original color as `image[sr][sc]`

Once we find such a pixel, we recolor it.

This is exactly what **DFS** or **BFS** is good for: explore all connected nodes in a component.

A key edge case:

```
if original_color == color:
    return image
```

If the starting pixel already has the target color, we should stop immediately. Otherwise, we may repeatedly revisit the same pixels unnecessarily.

2. Approach

We can use iterative BFS with a queue.

Pseudocode

```
floodFill(image, sr, sc, color):

    m = number of rows
    n = number of columns

    original_color = image[sr][sc]

    if original_color == color:
        return image

    create a queue
    add (sr, sc) to the queue

    while queue is not empty:
        remove the front cell (r, c)

        if image[r][c] is not original_color:
            continue

        recolor image[r][c] to color

        for each of the 4 directions:
            new_r = r + dr
            new_c = c + dc

            if new position is inside the grid:
                if image[new_r][new_c] == original_color:
                    add (new_r, new_c) to the queue

    return image
```

3. Python Solution

```
from typing import List
from collections import deque

class Solution:
    def floodFill(
        self,
        image: List[List[int]],
        sr: int,
        sc: int,
        color: int
    ) -> List[List[int]]:

        # Number of rows and columns in the image
        m = len(image)
        n = len(image[0])

        # The color of the starting pixel.
        # Only pixels with this color should be changed.
        original_color = image[sr][sc]

        # Important edge case:
        # If the new color is the same as the original color,
        # there is nothing to change.
        #
        # Without this check, we may keep adding already-colored
        # neighbors again and again.
        if original_color == color:
            return image

        # BFS queue starts from the given starting pixel.
        queue = deque()
```

```

queue.append((sr, sc))

# Four possible directions:
# up, down, left, right
directions = [
    (-1, 0), # up
    (1, 0), # down
    (0, -1), # left
    (0, 1) # right
]

# Process all reachable pixels with the original color.
while queue:
    r, c = queue.popleft()

    # If this pixel no longer has the original color,
    # skip it.
    #
    # This protects us from processing the same cell twice.
    if image[r][c] != original_color:
        continue

    # Recolor the current pixel.
    image[r][c] = color

    # Explore all 4 neighboring pixels.
    for dr, dc in directions:
        nr = r + dr
        nc = c + dc

        # Check if the neighbor is inside the grid.
        if 0 <= nr < m and 0 <= nc < n:
            # We only want to visit pixels that match
            # the starting pixel's original color.
            if image[nr][nc] == original_color:
                queue.append((nr, nc))

# Return the modified image.
return image

```

4. Complexity Analysis

Let:

- m be the number of rows
- n be the number of columns

Time Complexity

$O(m * n)$

In the worst case, every pixel in the image has the same color as the starting pixel, so we may visit every cell once. Each cell checks at most 4 neighbors, which is constant work.

Therefore, total time is:

$O(4 * m * n) = O(m * n)$

Space Complexity

$O(m * n)$

In the worst case, the BFS queue can contain many cells.

For example, if the entire image is one connected component of the same color, the queue may grow proportional to the number of pixels.

So the auxiliary space is:

$O(m * n)$

5. Alternative Approaches

Recursive DFS

We can also solve this using recursion.

```
class Solution:
    def floodFill(
        self,
        image: List[List[int]],
        sr: int,
        sc: int,
        color: int
    ) -> List[List[int]]:

        m = len(image)
        n = len(image[0])

        original_color = image[sr][sc]

        if original_color == color:
            return image

        def dfs(r: int, c: int) -> None:
            # Boundary check
            if r < 0 or r >= m or c < 0 or c >= n:
                return

            # Only fill pixels with the original color
            if image[r][c] != original_color:
                return

            # Recolor current pixel
            image[r][c] = color

            # Visit 4-directional neighbors
            dfs(r - 1, c)
            dfs(r + 1, c)
            dfs(r, c - 1)
            dfs(r, c + 1)

        dfs(sr, sc)
        return image
```

Recursive DFS is simpler, but for larger grids it can risk hitting Python's recursion limit. Since this problem has small constraints, it is usually accepted.

Iterative DFS

Instead of a queue, we can use a stack:

```
stack = [(sr, sc)]
```

This works the same way as BFS, except it explores deeply before exploring broadly.

Both BFS and DFS are valid because the problem only asks us to recolor the connected component, not to find shortest paths.